

Aula 26 - Programação Reativa com Spring WebFlux (Mono e Flux)

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Paradigma Reativo (15 min)

Problema do Modelo Tradicional (Blocking I/O)

- Thread por requisição → Alto consumo de recursos.
- Ineficiência em operações I/O (ex: chamadas a APIs externas).

Vantagens do Modelo Reativo

- **Não-blocking:** Threads não ficam ociosas esperando I/O.
- **Backpressure:** Controle do fluxo de dados (evita sobrecarga).

2. Flux vs Mono (25 min)

Tipo	Descrição	Exemplo de Uso
Mono	Emite 0 ou 1 elemento	Resposta de uma requisição HTTP
Flux	Emite 0 a N elementos (stream)	Listagem de produtos em tempo real

Operadores Comuns

```
java
Flux.range(1, 5)
    .map(i -> i * 2)           // Transformação
    .filter(i -> i > 5)       // Filtro
    .subscribe(System.out::println); // Consumidor
```

3. Spring WebFlux (20 min)

- **Stack Não-Blocking:**
 - **Netty** como servidor padrão.

- R2DBC para acesso reativo a bancos SQL.
- Anotações:
 - `@RestController` → `@RestController` (mesma anotação, comportamento diferente).

Parte Prática (3 horas)

Atividade: API de Produtos Reativa

Passo 1: Configuração (30 min)

1. Novo Projeto Spring Boot:

bash

```
spring init --dependencies=webflux,data-r2dbc,h2 reactive-api
```

2. Configuração do R2DBC (application.yml):

yaml

```
spring:
  r2dbc:
    url: r2dbc:h2:mem:///testdb
    username: sa
    password: ""
```

Passo 2: Entidade e Repository (1h)

1. Classe `Product` :

java

```
@Data @NoArgsConstructor @AllArgsConstructor
public class Product {
    @Id
    private Long id;
    private String name;
    private double price;
}
```

2. Repository Reativo:

java

```
public interface ProductRepository extends ReactiveCrudRepository<Product, Long> {
    Flux<Product> findByPriceGreaterThan(double price);
}
```

```
}
```

Passo 3: Controller Reativo (1h)

1. Endpoints Básicos:

```
java
```

```
@RestController
@RequestMapping("/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductRepository repository;

    @GetMapping
    public Flux<Product> getAll() {
        return repository.findAll();
    }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable Long id) {
        return repository.findById(id);
    }
}
```

2. Endpoint com Stream em Tempo Real:

```
java
```

```
@GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<Product> streamProducts() {
    return repository.findAll()
        .delayElements(Duration.ofSeconds(1)); // Simula dados chegando em tempo real
}
}
```

Passo 4: Testes (30 min)

1. Teste com WebTestClient:

```
java
```

```
@SpringBootTest
class ProductControllerTest {
    @Autowired
    private WebTestClient webClient;

    @Test
    void shouldReturnProducts() {
        webClient.get().uri("/products")
    }
}
```

```
        .exchange()
        .expectStatus().isOk()
        .expectBodyList(Product.class).hasSize(2);
    }
}
```

2. Teste do Stream:

```
java

@Test
void shouldStreamProducts() {
    webClient.get().uri("/products/stream")
        .exchange()
        .expectStatus().isOk()
        .expectHeader().contentType(MediaType.TEXT_EVENT_STREAM)
        .returnResult(Product.class);
}
```

Tarefa para Casa

1. Integrar com MongoDB Reativo:

- Substituir R2DBC por `spring-boot-starter-data-mongodb-reactive`.

2. Criar um Client Reactivo:

- Usar `WebClient` para consumir outra API reativa.

Próxima Aula

- **Tópico:** Segurança em APIs Reativas (Spring Security + WebFlux).
- **Atividade prática:** Autenticação JWT em endpoints reativos.

Resultado Esperado

- API reativa funcional com:
 - Operações CRUD não-blocking.
 - Stream de dados em tempo real.
 - 100% de cobertura de testes.

- ✓ Endpoints reativos (Mono/Flux)
- ✓ Banco de dados não-blocking (R2DBC)
- ✓ Pronto para segurança reativa

<https://projectreactor.io/docs/core/release/reference/images/flux.png> (*Flux em ação - Project Reactor*) 🚀